

UNIVERSITY INSTITUTE OF COMPUTING • CHANDIGARH UNIVERSITY
MOHALI, PUNJAB

ACADEMIC PROJECT REPORT

SMART HOSPITAL MANAGEMENT SYSTEM

Patient Records • Emergency Triage • Bed Allocation • Discharge Billing

Computer Programming | Course Code 26CAH-101
Academic Year 2026–2027

SUBMITTED BY
Palash Behura
UID: 26BCA10216
Programme: BCA
Section: BCA-2(A)

SUBMITTED TO
Course Instructor
Computer Programming (26CAH-101)
Department of Computing
Chandigarh University

Acknowledgement

I would like to thank the faculty teaching Computer Programming (26CAH-101) for the guidance and feedback that supported the development of this project. Working on the Smart Hospital Management System helped me apply C programming to a connected problem involving patient records, emergency priority, bed availability and billing.

I am also grateful to Chandigarh University for the learning environment and computing resources, and to my family and friends for their encouragement during the project.

Palash Behura | UID: 26BCA10216 | BCA-2(A)

Abstract

The Smart Hospital Management System is a menu-driven console application written in C. It brings together patient registration, emergency-first triage, doctor assignment, bed allocation, treatment-charge entry, patient search and updates, discharge billing, and a hospital summary. The program stores patient records in patients.dat and the three-ward, thirty-bed occupancy grid in beds.dat. When an admitted patient is discharged, the program calculates the bill using a recursive function and writes a discharge_<id>.txt summary.

The implementation uses structures to represent patients, doctors and charge details; arrays to track beds and charge components; dynamic memory for patient lists; pointer-based record access; and binary file operations for persistence. The application is a learning prototype rather than a production hospital system. This report documents the implemented workflow, design, sample output, test evidence and known limitations.

KEYWORDS **Patient records • Emergency triage • C programming • Binary files • Recursion • Bed allocation**

Table of Contents

Acknowledgement and Abstract	2
Table of Contents	3
1. Introduction	4
2. Objectives and Project Scope	4
3. Program Workflow and Flowchart	5
4. System Design	6
5. Function Overview	7
6. Core Implementation Details	8
7. Console Output	9–10
8. Testing and Verification	11
9. Limitations and Future Scope	11
10. Build Instructions and Conclusion	12
References	12

The report follows the supplied C implementation and separates observed runtime results from limitations that still need to be addressed.

1. Introduction

Hospital administration involves several linked tasks: recording patient details, identifying urgent cases, assigning doctors, tracking available beds and preparing treatment bills. This project models a small subset of those workflows in a single console application, so the data and decisions can be followed through the program.

The application is designed for demonstration and coursework. It uses a numbered menu to call individual C functions. Patient records are retained in a binary file, bed occupancy is stored separately, and a plain-text discharge summary is generated for a discharged patient. Because the two binary files and in-memory doctor counts are managed separately, consistent state handling is an important design concern.

2. Objectives and Project Scope

- Register patients with an automatically assigned record ID and basic input validation.
- List active patients with emergency-priority records displayed before normal-priority records.
- Assign patients to one of five preconfigured doctors and display a three-ward bed grid.
- Record consultation, medicine, room-per-day and test charges, then calculate a discharge bill.
- Support patient search, selected record updates, full patient listing and a summary dashboard.
- Preserve patient and bed data between runs and create a text file for each completed discharge.

Main Menu Functions

Area	Implemented behaviour
Patient administration	Register, search, update and display records.
Triage and assignment	Sort by priority and assign a configured doctor.
Beds and admission	View occupancy and select the first free bed in a chosen ward.
Charges and discharge	Store cost components, release a bed and write a bill summary.
Oversight	Summarise records, admitted patients, emergency-priority records, free beds and discharged bills.

3. Program Workflow

1. Initialise the five doctor records and load bed occupancy from beds.dat. If that file is unavailable, initialise all thirty beds as free.
2. Display the main menu and read the selected option inside a do-while loop.
3. Call the function associated with the selection: registration, triage, doctor assignment, beds, charges, discharge, search, update, display or summary.
4. For operations that use patient collections, read records from patients.dat and allocate temporary memory where required.
5. Update the stored patient record using its file position, or append a new record during registration.
6. On discharge, release the saved bed, sum four bill components recursively, mark the patient as discharged and write discharge_<id>.txt.
7. Return to the menu after each operation. Exit when the selected option is 0.

Program Flowchart

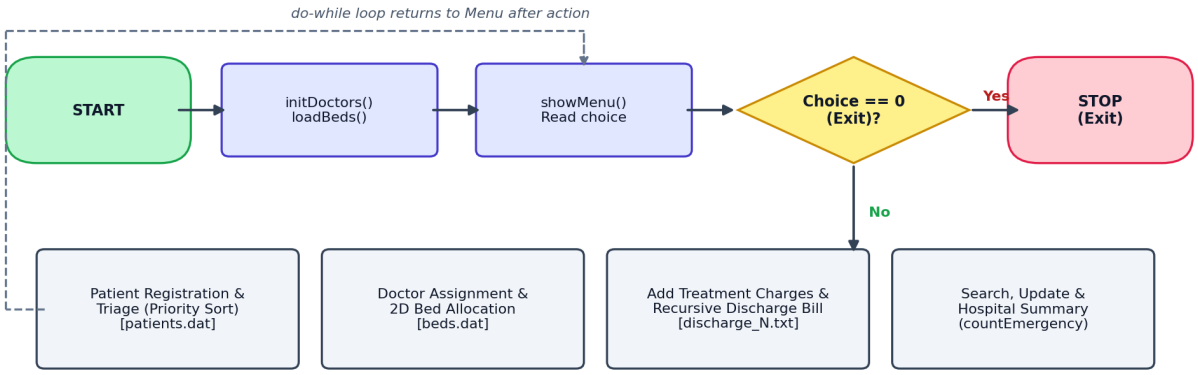


Figure 1. High-level control flow, menu decision and main operation groups.

The hidden option 99 loads demonstration records by deleting and recreating patients.dat and resetting beds.dat. Use it only in a disposable working folder.

4. System Design

The program keeps the source compact by defining three structures and using a two-dimensional integer array for bed occupancy. Status and priority are stored as integer codes, while doctor and bed references use -1 until they are assigned.

Structure	Key fields	Purpose
struct Patient	id, name, age, gender, disease, priority, doctorId, wardNo, bedNo, daysAdmitted, status and nested charges.	Primary patient record written to patients.dat.
struct Charges	consultation, medicine, roomPerDay and tests (float values).	Stores the four components used in discharge billing.
struct Doctor	id, name, department and patientCount.	Five doctor records are initialised at program start; patientCount remains in memory.

4.1 Bed Model and Record States

Item	Representation in code
Wards	3: General, ICU and Pediatric
Capacity	10 beds per ward; 30 total
Bed values	0 = free; 1 = occupied
Patient priority	1 = Emergency; 2 = Normal
Patient status	0 = Waiting; 1 = Admitted; 2 = Discharged
Unassigned references	doctorId, wardNo and bedNo use -1

4.2 File Storage

File	Format	Role
patients.dat	Binary records	Appends a complete struct Patient and updates records in place with fseek() and fwrite().
beds.dat	Binary grid	Stores the full int beds[3][10] array.
discharge_<id>.txt	Plain text	Stores identifying details, priority, days admitted, charges and total bill.

The binary format depends on the compiler and the current layout of struct Patient. These files are suitable for this single-user prototype, not as a portable database format.

5. Function Overview

The menu calls small functions for specific operations. This separation makes the workflow easier to read and allows the important data-handling steps to be discussed independently.

Function / group	Responsibility
initDoctors()	Loads the five configured doctors and sets their in-memory patient counts to zero.
loadBeds() / saveBeds()	Reads or writes the 3 × 10 occupancy array in beds.dat.
registerPatient()	Collects patient information, validates age, gender and priority, sets defaults and appends the record.
triageList()	Loads patient records into allocated memory, bubble-sorts by priority, shows non-discharged patients and frees the array.
assignDoctor()	Checks doctor ID and the current in-memory load before updating the patient record.
showBeds() / allocateBed()	Displays ward occupancy and marks the first free bed in the selected ward.
addCharges() / dischargePatient()	Updates charge data; on discharge, releases the bed, totals the bill and creates a text summary.
searchPatient() / updatePatient()	Finds records by ID or case-sensitive name substring and updates selected fields.
displayAll() / hospitalSummary()	Prints all saved records or summary values including admitted count, free beds and discharged bills.
totalBill() / countEmergency()	Recursive helpers for charge summation and emergency-priority counting.
loadDemoPatients()	Replaces patient records with five demonstration records; use only with disposable data.
main()	Initialises state, shows the menu, dispatches operations through switch and loops until exit.

6. Core Implementation Details

6.1 Record Access and File Positioning

Patient records are stored sequentially as binary structures. When an existing record is found, the program tracks the record start position and seeks back before writing the modified structure. This is used by doctor assignment, charge updates, patient updates and discharge.

```
/* Update a record at its saved byte position */
fseek(f, pos, SEEK_SET);
fwrite(&rec, sizeof(struct Patient), 1, f);
```

The supplied implementation does not check every fread(), fwrite() or fseek() return value. The file update can therefore fail without every path reporting that failure.

6.2 Pointer-Based Traversal and Dynamic Memory

Several functions allocate an array of Patient records using malloc() or calloc(). Pointer arithmetic such as arr + i addresses an individual record, and the arrow operator (->) reads its fields. The arrays are released with free() after use. showBeds() receives a pointer to an array of ten integers, allowing it to traverse each ward.

```
struct Patient *cur = arr + i;
if (cur->priority == 1) { /* Emergency */ }

/* 2D bed grid: one row contains BEDS_PER_WARD integers */
void showBeds(int (*grid)[BEDS_PER_WARD]);
```

6.3 Recursive Calculations

totalBill() stops when n equals zero and otherwise adds charges[n - 1] to the result for the remaining entries. countEmergency() follows the same pattern: it checks the last patient in the current range and adds one when that priority is 1.

```
float totalBill(float charges[], int n) {
    if (n == 0) return 0.0f;
    return charges[n - 1] + totalBill(charges, n - 1);
}

int countEmergency(struct Patient *arr, int n) {
    if (n == 0) return 0;
    int hit = ((arr + n - 1)->priority == 1) ? 1 : 0;
    return hit + countEmergency(arr, n - 1);
}
```

Both functions use a clear base case and reduce n on every recursive call. The summary currently counts Emergency priority across all saved records, including discharged patients.

7. Console Output

These panels were rendered from captured stdout of the supplied program compiled with GCC. The demo sequence uses the program's built-in records; the values are examples, not real patient information.

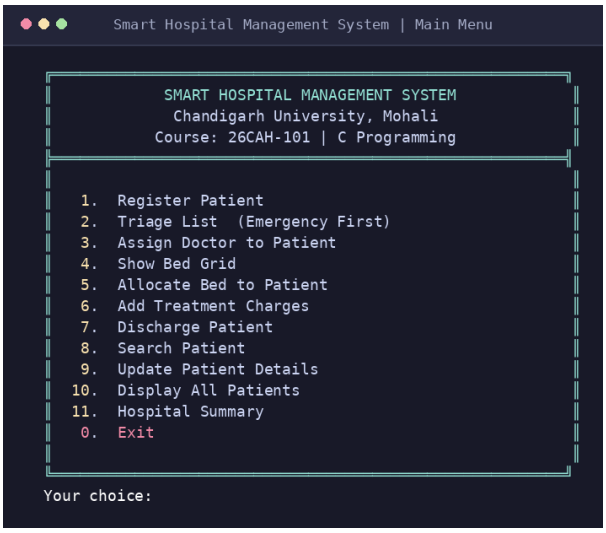


Figure 2. Main menu and operation list.



Figure 3. Emergency-priority patients appear first.

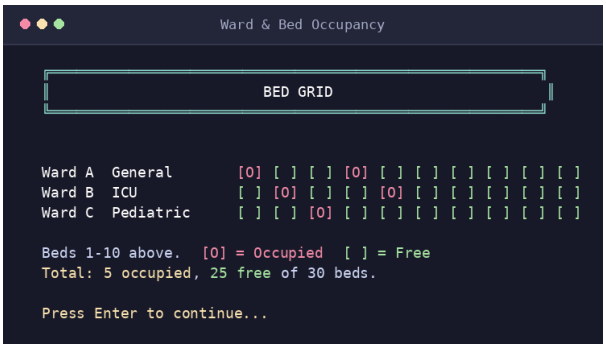


Figure 4. Three wards and thirty bed positions.

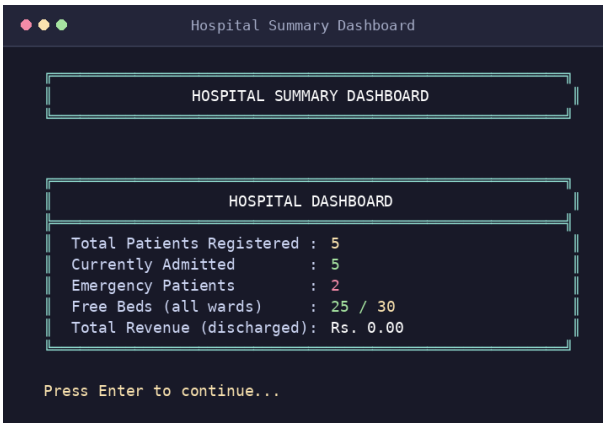


Figure 5. Summary dashboard for the demo records.

Observed demo-state values

With option 99 in a disposable folder, the program loads five demonstration patients. The triage list shows priority 1 before priority 2, and the initial bed grid contains five occupied beds and twenty-five free beds.

7. Console Output (continued)

The discharge workflow releases the allocated bed, marks the patient record as discharged, calculates the total recursively and writes a discharge summary text file.

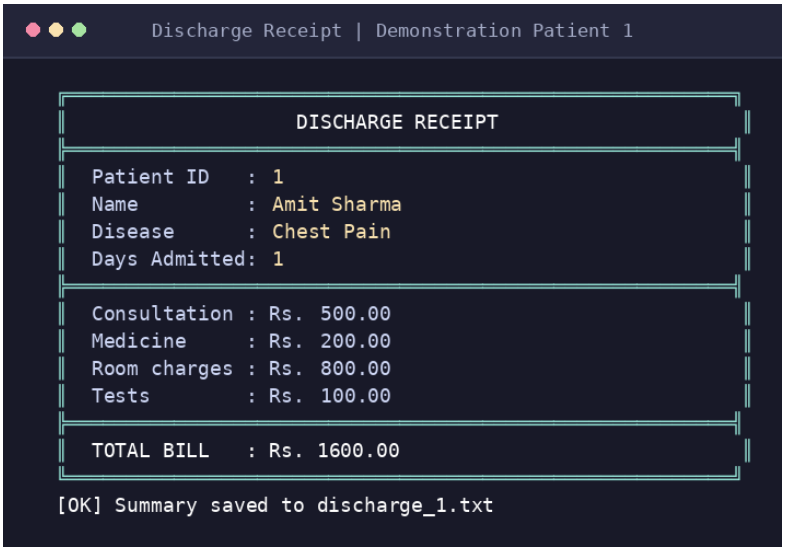


Figure 6. Discharge receipt generated from the demo run.

DEMO BILL

Patient: Amit Sharma
Patient ID: 1
Consultation: Rs. 500.00
Medicine: Rs. 200.00
Room: Rs. 800.00
Tests: Rs. 100.00

TOTAL Rs. 1,600.00

The run also created discharge_1.txt in its temporary working directory.

Example bill calculation

Component	Amount
Consultation	Rs. 500.00
Medicine	Rs. 200.00
Room: Rs. 800 × 1 day	Rs. 800.00
Tests	Rs. 100.00
Total	Rs. 1,600.00

The displayed amount follows the demonstration values currently assigned in loadDemoPatients(); actual amounts depend on the charges entered for a patient.

8. Testing and Verification

The C source was compiled and a smoke scenario was run in a temporary working directory. The checks covered the built-in demo loader, triage, bed grid, summary, discharge receipt, summary file creation and exit. The test used disposable data so it did not overwrite the user's uploaded records.

Check	Scenario	Observed result	Status
Build	gcc -std=c99 -Wall -Wextra -Wpedantic src/main.c -o hospital	Build completed; two format-overflow warnings remain.	Built
Demo loader	Option 99 in a temporary folder	Five demonstration records loaded.	Pass
Emergency-first triage	Option 2 after loading demo data	Emergency records displayed before normal records.	Pass
Bed grid	Option 4 after loading demo data	5 occupied and 25 free beds out of 30.	Pass
Hospital summary	Option 11 after loading demo data	Shows 5 records and 2 Emergency-priority records.	Pass
Discharge	Option 7; discharge patient ID 1	Bill is Rs. 1,600.00 and discharge_1.txt is created.	Pass
Exit	Option 0	Application prints the exit message.	Pass

Compiler note: GCC warns that sprintf() may write a long patient name or disease string into the 60-byte temporary buffer used only for receipt-line width calculations. Replace those calls with safe length calculations or suitably bounded formatting before treating the build as warning-free.

9. Limitations and Future Scope

- Doctor patientCount values are reset at startup and are not reconstructed from saved patient records; capacity checks can become inaccurate after a restart.
- Bed allocation marks a bed occupied before verifying that the requested patient exists and is eligible, which can desynchronise the bed grid.
- Option 99 deletes and recreates patients.dat; it must only be used with disposable data.
- Not every file operation and numeric input is checked robustly. The program has no authentication, concurrent-user protection or transaction recovery.
- Future work should validate a patient before bed allocation, rebuild doctor loads, check all I/O results, improve input handling and move persistent records to a database.

10. Build Instructions and Conclusion

10.1 Compile and Run

Requirements: a C compiler with C99 support (for example, GCC) and a UTF-8 terminal for the box-drawing characters and ANSI colours.

```
# Compile from the project root
gcc -std=c99 -Wall -Wextra src/main.c -o hospital

# Run on macOS / Linux
./hospital

# Run in Windows Command Prompt (MinGW)
hospital.exe
```

Runtime files are created in the current working directory: patients.dat, beds.dat and discharge_<id>.txt. Back up patient and bed files before testing. Menu option 99 overwrites patient data and should only be used in a disposable folder.

11. Conclusion

The Smart Hospital Management System demonstrates how a small C application can connect record management, emergency-priority sorting, doctor assignment, bed occupancy and discharge billing. Its structures organise the patient and charge data, file handling preserves records across runs, and recursive functions calculate bills and count Emergency-priority records. The console output makes the main actions visible and gives a clear basis for demonstrating the program in an academic presentation.

The smoke run confirms that the principal demonstration flow works, including the creation of a discharge summary. The known compiler warnings and data-consistency limitations should be addressed before the application is extended or used with important records.

References

[1] Smart Hospital Management System with Emergency Triage, supplied C source (src/main.c).

[2] ISO/IEC 9899:2018, Programming Languages — C.

[3] E. Balagurusamy, Programming in ANSI C, McGraw Hill Education.

[4] Chandigarh University, “Our Identity / University Seal,” https://www.cuchd.in/about/our-identity.php (accessed 27 October 2026).

Academic note: this report describes the supplied implementation and recorded smoke run. It does not claim that the project is suitable for real clinical operations.